Dockerized Sports Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Priyanshi Gandhe	EN22CS301764
2	Priyanshu Sengar	EN22CS301770
3	Rudra Yadav	EN22CS301831
4	Prithviraj Singh Tomar	EN22CS301757
5	Pranjali Sharma	EN22CS301733

Group Name: 07D7

Project Number: D0-30

Industry Mentor Name:

University Mentor Name: Prof. Shyam Patel

Academic Year: 2026

INDEX

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Future Scope & Enhancements

1. Problem Statement & Objectives

1.1 Problem Statement

Containerize a Sports service using industry best practices for Docker. The project includes writing multi-stage Dockerfiles to optimize image size and security, alongside a Docker Compose configuration to orchestrate the application and its database dependencies. This ensures a consistent, portable development and production environment across the entire team.

In modern software development, applications must be deployed quickly, reliably, and consistently across different environments such as local machines, cloud servers, and production systems. However, developers often face challenges such as dependency mismatches, inconsistent runtime environments, and configuration errors when deploying applications on different machines.

For a **Sports Service application**, these challenges become more critical when the application is deployed on cloud infrastructure such as **Amazon Web Services**. Even if the application works correctly on a developer's machine, it may fail on a cloud server due to missing dependencies, configuration issues, or incompatible environments.

Initially, while deploying the application on the AWS server, the application was not functioning correctly due to environment configuration issues. These problems highlighted the need for a standardized and automated deployment solution.

To address these challenges, containerization and orchestration technologies are required. Using **Docker**, the application can be packaged along with all its dependencies into a container image that runs consistently on any system. However, managing containers at scale requires orchestration tools such as **Kubernetes**, which help automate deployment, scaling, and management of containers.

In addition, implementing a Continuous Integration and Continuous Deployment pipeline using **GitHub Actions** ensures that application updates are automatically built, tested, and deployed without manual intervention.

Therefore, the problem addressed in this project is creating a **reliable, portable, and automated deployment architecture for a sports service application using containerization, orchestration, and CI/CD automation**.

1.2 Project Objectives

The primary objective of this project is to design and implement a **containerized and automated deployment system for a sports service application** using modern DevOps tools.

The key objectives include:

- To containerize the sports service application using Docker.
- To create a **Dockerfile** that defines the application environment and dependencies.

- To build and manage **Docker images and containers** for consistent application execution.
- To deploy the application on a cloud server using AWS.
- To integrate **Kubernetes for container orchestration and scalability**.
- To implement an automated **CI/CD pipeline using GitHub Actions**.
- To ensure reliable and consistent deployment across development and production environments.
- To improve application scalability, availability, and maintainability.

1.3 Scope of the Project

The scope of this project focuses on **containerization, orchestration, and automated deployment of the sports service application**.

The system includes building the application into a Docker image, running it as a container, deploying it on a cloud server, and managing the deployment using Kubernetes.

This project focuses on:

- Docker-based application containerization
- Creation and management of Docker images and containers
- Cloud deployment on AWS server
- Kubernetes-based container orchestration
- CI/CD automation using GitHub Actions
- Automated build and deployment workflow
- Environment-based configuration management

This project mainly focuses on **deployment architecture and DevOps practices**, rather than the internal implementation of sports-related business logic.

2. Proposed Solution

2.1 Key Features

The Dockerized Sports Service system provides the following major features:

- Containerized application deployment using Docker
- Portable Docker images for consistent execution
- Cloud deployment using AWS infrastructure
- Kubernetes-based container orchestration
- Automated CI/CD pipeline using GitHub Actions
- Automatic build and deployment process
- Improved scalability and reliability
- Simplified development and deployment workflow

2.2 Overall Architecture / Workflow

The system architecture follows a **modern DevOps deployment pipeline** that integrates containerization, orchestration, and automation.

The workflow begins when developers push the application code to a GitHub repository. Once the code is pushed, the CI/CD pipeline configured using GitHub Actions is triggered automatically.

The pipeline performs the following steps:

1. The application code is fetched from the GitHub repository.
2. A Docker image is built using the Dockerfile.
3. The Docker image is pushed to a container registry.
4. Kubernetes pulls the image and deploys it inside Pods.
5. Deployments manage the Pods and ensure the required number of instances are always running.
6. Kubernetes Services expose the application so users can access it.

During the initial deployment stage, the application faced issues while running on the AWS server due to configuration errors. These issues were resolved by properly configuring the container environment and deployment setup.

By integrating Kubernetes, the system can automatically restart failed containers, scale application instances based on demand, and maintain high availability.

The CI/CD pipeline ensures that every new update to the application is automatically built and deployed, reducing manual effort and minimizing deployment errors.

2.3 Tools & Technologies Used

The tools and technologies used in this project include:

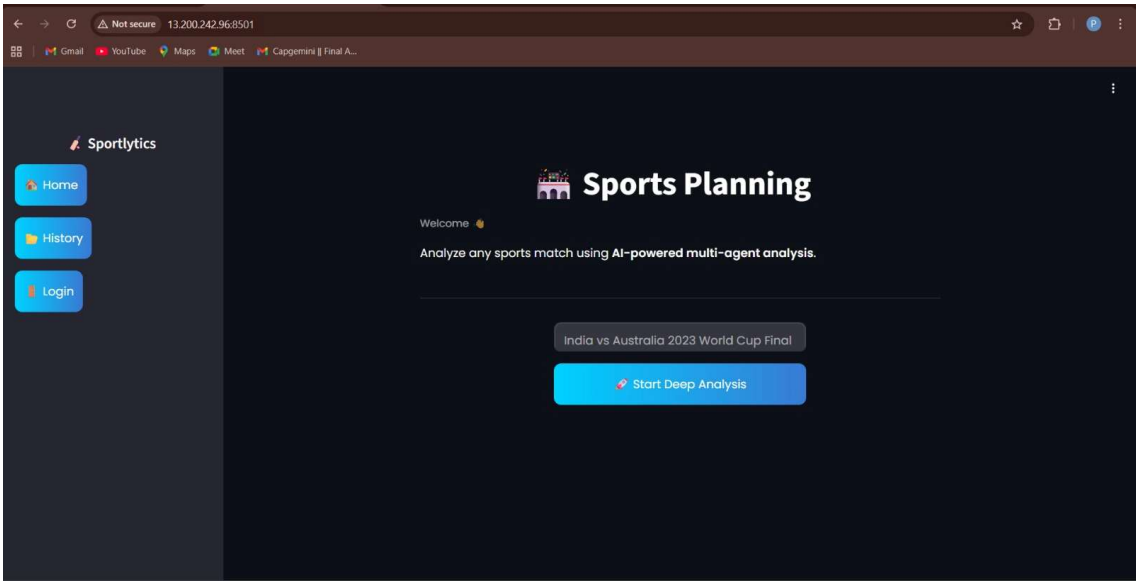
- Programming Language: Application Backend Technology
- Containerization Platform: Docker
- Container Orchestration: Kubernetes
- Cloud Platform: Amazon Web Services (AWS)
- CI/CD Automation: GitHub Actions
- Version Control System: Git
- Container Image Management: Docker Images
- Deployment Infrastructure: Kubernetes Pods, Deployments, Services
- Configuration Management: Environment Variables (.env)

These tools together create a modern and scalable **DevOps deployment pipeline**.

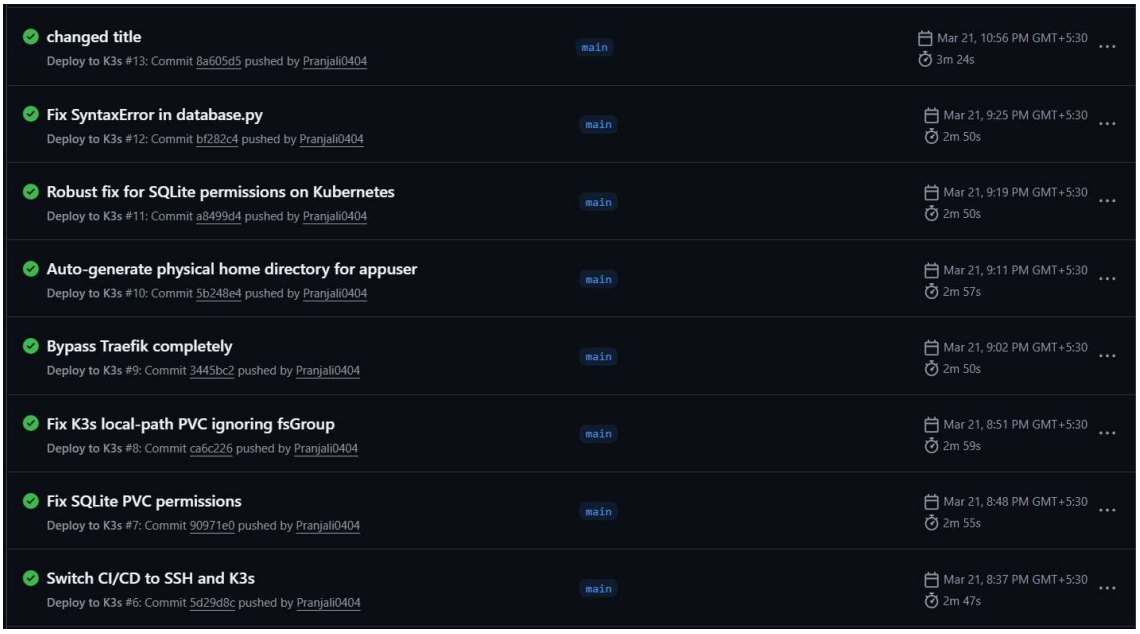
3. Results & Output

3.1 Screenshots / Outputs

- Application running on AWS server



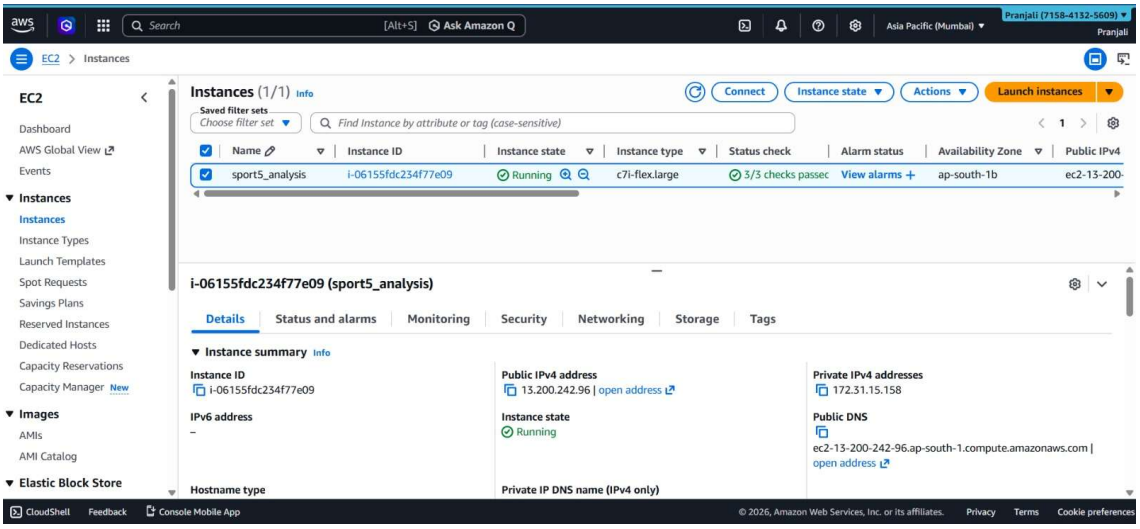
- CI/CD pipeline execution



4. Reports / Dashboards / Models

Although the project primarily focuses on deployment infrastructure, it produces useful technical outputs such as:

- EC2 Instance



- Container logs

```
ubuntu@ip-172-31-15-158:~$ docker logs 364b9521b3c9

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.17.0.2:8501
  External URL: http://65.0.107.238:8501
```

- Kubernetes Deployment Logs

```
ubuntu@ip-172-31-15-158:~$ sudo kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
sportlytics-deployment-cb949887c-8ds4p  1/1     Running   0          33h
```

- Docker images

```
ubuntu@ip-172-31-15-158:~$ sudo docker ps -a
CONTAINER ID   IMAGE             COMMAND                  CREATED       STATUS                     PORTS     NAMES
364b9521b3c9   my-streamlit-app  "streamlit run app.p…"   6 days ago    Exited (0) 5 days ago              stupefied_darwin
ubuntu@ip-172-31-15-158:~$ sudo docker images
REPOSITORY                      TAG         IMAGE ID       CREATED       SIZE
final_agentic_app               latest      55d4fdcea865   4 days ago    1.32GB
<none>                          <none>      030b08497b34   4 days ago    549MB
my-streamlit-app                latest      791e9d890787   8 days ago    1.68GB
pranjalisharma29/sports-planning latest     791e9d890787   8 days ago    1.68GB
python                          3.10-slim   f2d628de8a5f   9 days ago    122MB
ubuntu@ip-172-31-15-158:~$
```

5. Key Outcomes

The major outcomes achieved through this project include:

- The sports service application is successfully containerized using Docker.
- The application can run consistently across different environments.
- Deployment issues on the AWS server were identified and resolved.
- Kubernetes enables automated container management and scaling.
- GitHub Actions automates the build and deployment workflow.
- The CI/CD pipeline reduces manual deployment effort.
- The system architecture becomes more scalable, reliable, and maintainable.

Overall, the project successfully demonstrates a **modern DevOps deployment approach for containerized applications**.

6. Conclusion

The **Dockerized Sports Service with Kubernetes and CI/CD pipeline** demonstrates a modern approach to application deployment using containerization, orchestration, and automation.

Docker ensures that the application runs consistently across different environments by packaging the application and its dependencies into containers. Kubernetes enhances the system by managing container deployment, scaling, and recovery automatically.

The integration of GitHub Actions introduces an automated CI/CD pipeline that builds and deploys the application whenever new code changes are pushed to the repository.

This architecture significantly improves deployment reliability, reduces manual intervention, and enables scalable cloud-based application management.

The project successfully achieves its goal of creating a **portable, scalable, and automated deployment system for a sports service application**.

7. Future Scope & Enhancements

The project can be further enhanced by incorporating additional production-grade features such as:

- Advanced CI/CD pipelines with automated testing
- Load balancing using NGINX or Ingress controllers
- Kubernetes Helm charts for simplified deployment
- Monitoring and observability using Prometheus and Grafana

- Centralized logging using ELK Stack
- Auto-scaling using Kubernetes Horizontal Pod Autoscaler
- Secure secrets management using AWS Secrets Manager
- Integration with additional cloud services

These enhancements

would make the system even more suitable for **enterprise-scale deployment**.

8. Expected Impact

The expected impact of this project is significant in improving software deployment and development workflows. By using containerization and orchestration technologies, developers can work in consistent environments and avoid dependency-related issues.

The automated CI/CD pipeline ensures faster and more reliable deployments, reducing manual errors and improving productivity.

In production environments, the use of Kubernetes allows the system to scale easily and maintain high availability. Overall, the project contributes to building a **robust, scalable, and efficient deployment infrastructure for modern applications**.